STL library

BIE-PA2 – Programming and Algorithmic II

Ladislav Vagner

Department of Theoretical Computer Science
Faculty of Information Technology
Czech Technical University in Prague
xvagner@fit.cvut.cz

March 16, 2026

Overview

- Sample programs with STL:
  - aggregation, I/O simplification,
  - containers, median, `vector`,
  - remove duplicates, `set`.
- Concepts:
  - generic classes, auxiliary classes,
  - iterators,
  - functors.
- Lambda functions.
- Basic containers:
  - `array`, `vector`,
  - `deque`, `list`,
  - `set`, `map`.
- Algorithm library.

Motivation

We will present some simple programs to demonstrate basic use of STL:

- sum numbers from the input,
- find median,
- remove duplicate words.

The examples will introduce concepts that are common to all programs using STL.

Sum input numbers

A traditional C++ solution is a program like:

```
#include <iostream>
using namespace std;

int main ( )
{
  int sum = 0, x;
  while ( cin >> x )
    sum += x;
  cout << sum << endl;
  return 0;
}
```

Sum input numbers - STL

The same problem solved with STL:

```
#include <iostream>
#include <numeric>
#include <iterator>
using namespace std;

int main ( )
{
  cout << accumulate ( istream_iterator<int> ( cin ),
                       istream_iterator<int> (), 0 )
       << endl;
  return 0;
}
```

Sum input numbers - STL

- There are two basic concepts used in the example: iterator and generic functions. An iterator:
  - is a data type declared in STL,
  - references some data or position,
  - the data may be located in some collection (vector, set, ...), or in an input file,
  - the sample iterator reads integers from the given input stream (`cin`),
  - iterators are usually generic classes,
  - iterator interface supports dereference (*), move forward (++), and comparison of two iterators (==, !=),
  - some iterators add further methods/operators.
- `istream_iterator<int> ( cin )` constructs new input iterator. This iterator reads input integer when dereferenced,
- `istream_iterator<int> ()` is a dummy object meaning we shall iterate until EOF is reached.

Sum input numbers - STL

- Generic function `accumulate`:
  - is a generic defined in STL,
  - it sums values in the range defined by two iterators
  - the function passes the input range,
  - the values read from the iterators are accumulated to the result (operator +),
  - the initial value is 0 (given by the last parameter),
  - return value is the total sum.
- There are many similar functions in STL, the functions:
  - copy ranges,
  - erase ranges,
  - filter data,
  - sort,
  - ...

Product of input numbers

STL solution:

```
#include <iostream>
#include <numeric>
#include <iterator>
using namespace std;

int multiply ( int a, int b )
{
  return a * b;
}

int main ( )
{
  cout << accumulate ( istream_iterator<int> ( cin ),
                       istream_iterator<int> (), 1, multiply )
       << endl;
  return 0;
}
```

Product of input numbers

- Generic function `accumulate` is overloaded in STL:
  - the general variant accepts the operation to replace operator +,
  - we passed our own function `multiply`, this function combines the result by multiplication,
  - the function is called for each input value,
  - the function is called with the accumulated value and the value read in its parameters,
  - return value is the updated accumulated value,
  - initial value was changed to 1.
- Other aggregated values (maximum, minimum, ...) can be computed with `accumulate`, ...
- A function passed to a generic STL function is a common trick in STL. The last parameter accepts some "callable" value: a function, a function object (functor), or a lambda function (C++11).

Average of numbers in the input

```
#include <iostream>
#include <numeric>
#include <iterator>
using namespace std;

pair<int,int> avg ( const pair<int,int> & a, int b )
{
  return make_pair ( a . first + b, a . second + 1 );
}

int main ( )
{
  pair<int,int> x = accumulate ( istream_iterator<int> ( cin ),
                                 istream_iterator<int> (),
                                 make_pair(0,0), avg );
  cout << (double) x . first / x . second << endl;
  return 0;
}
```

Average of numbers in the input

- STL declares generic class `pair`. The class has two fields: `first` and `second`.
- Field data types are given in generic parameters, we declared two-integer pair.
- `pair` template is a convenient way to declare structure with just two fields. E.g., we need to store sum and count in our example.
- There is similar class `tuple` in C++ 11, a `tuple` may have arbitrary number of fields.
- `pair` and `tuple` is easy to declare. It sometimes leads to abuse of pairs and tuples. Field names are not very explanatory. It is often better to declare own type with more descriptive field names.

Average of numbers in the input

- An extension: compute the average of input numbers that are greater than some threshold:
  - avoid `accumulate`, replace it with a conventional loop,
  - but `accumulate` is convenient, e.g. when traversing complex structures like trees. Moreover, it is just one line of source code!
  - We just need to pass some "context" information (the threshold) to the `avg` function,
  - STL uses function objects (functors) to store the context information. The call invokes overloaded operator () in the object, the implementation does the computation,
  - function object (functor) – is an object that can be used like function. The class overloads operator (). Thus the object instance followed by parentheses (with parameters) invokes the overloaded operator – the object behaves like a function. Moreover, member variables may hold the context information needed for the computation.

Average of numbers in the input

```
#include <iostream>
#include <numeric>
#include <iterator>
#include <sstream>
using namespace std;
class avgThreshold
{
  int m_Threshold, m_Cnt, m_Sum;
public:
  avgThreshold ( int threshold )
     : m_Threshold ( threshold ), m_Cnt ( 0 ), m_Sum ( 0 ) { }
  double operator () ( double dummy, int x ) {
    if ( x > m_Threshold ) {
      m_Cnt ++; m_Sum += x;
    }
    return (double) m_Sum / m_Cnt;
  }
};
```

Average of numbers in the input

```
…

int main ( int argc, char * argv[] )
{
  int thr;
  if ( argc < 2 || ! (istringstream ( argv[1] ) >> thr) )
    return 1; // error
  cout << accumulate ( istream_iterator<int> ( cin ),
                       istream_iterator<int> (), 0.0,
                       avgThreshold ( thr ) )
       << endl;
  return 0;
}
```

Average of numbers in the input

- An instance of `avgThreshold` is passed to `accumulate`.
- The instance holds context (sum, count, and threshold).
- Operator `()` is invoked with each input number read.
- The instance of `avgThreshold` updates its member variables based on the input number.
- Return value is the actual average.
- Input parameter `dummy` is ignored (we do not use previous average in the computation).

Find median

Our program reads input integers and finds median value:

- the size of the input is not known,
- we need to store input numbers in an array to find median,
- sort the array,
- print out middle element.

We use "extensible" array - `vector`:

- `vector` implements dynamically allocated array,
- the methods increase array size when the capacity is exhausted.

Find median - STL

```
#include <iostream>
#include <vector>
#include <algorithm>
#include <iterator>
using namespace std;

int main ( )
{
  vector<int> numbers;
  for ( int x = 0; cin >> x; )
    numbers . push_back ( x );
  sort ( numbers . begin (), numbers . end () );
  cout << numbers [ numbers . size () / 2 ] << endl;
  return 0;
}
```

Find median - STL

```
#include <iostream>
#include <vector>
#include <algorithm>
#include <iterator>
using namespace std;


int main ( )
{
  vector<int> numbers;
  copy ( istream_iterator<int> ( cin ),
         istream_iterator<int> (), back_inserter ( numbers ) );
  sort ( numbers . begin (), numbers . end () );
  cout << numbers [ numbers . size () / 2 ] << endl;
  return 0;
}
```

Find median

STL function `copy` copies values from an input range (given by two iterators) to the destination (the third iterator):

- the problem is the, so far, empty destination,
- we want to append the values to the end,
- a specialized iterator `back_inserter` does the trick,
- this iterator does not overwrite existing values, instead, it adds the elements to the end (allocating new space as needed).

The sorting is done by `sort` function:

- the function sorts elements in the given input range,
- operator < is used to compare elements when sorting,
- the sorting order may be modified if custom function/functor is passed to `sort`.

The sample solution does not handle even number of input values (an additional condition solves the problem).

Remove duplicate words

The input is formed by words:

- our program reads input words,
- it filters out words it that has already seen,
- the words are displayed to standard output, with the input order preserved.

`set` data structure is convenient for this task:

- store input words in the set,
- if the word is not present in the set, display it,
- if the word is already present in the set, do not display it.

Container `vector` is not a good solution:

- we would have to search `vector` with each input word, time complexity would be $O(n^2)$,
- `set` decides present/not present in $O(\log n)$ time, time complexity will be $O(n \log n)$.

Remove duplicate words

```
#include <iostream>
#include <set>
#include <string>
using namespace std;

int main ( )
{
  set<string> seen;
  string str;

  while ( cin >> str )
    if ( seen . insert ( str ) . second )
      cout << str << endl;

  return 0;
}
```

Remove duplicate words

The program uses method `insert`:

- to store the word,
- to test the word was actually inserted to the set.

Method `insert` return value is:

```
pair<set<T>::iterator,bool>
```

- the first field is an iterator referencing the inserted value. We do not have any use for this information – we ignore this field,
- the second field is a flag indicating whether the value was actually inserted (`true`), or not (`false` – it has already been there).

Remove duplicate words

Modify the program to ignore upper/lower case:

- set is a good container to store the values,
- the default implementation does not fit our needs – it distinguishes lower/upper case strings,
- default implementation uses operator < to compare strings,
- set template has generic parameters to define custom comparison, set constructor accepts function/functor instance to actually compare the values.

Remove duplicate words

`set` template is declared as:

```
template <class Key,
          class Compare = less<Key>,
          class Allocator = allocator<Key> > set;
```

- generic parameter `Key` denotes the type stored,
- generic parameter `Compare` defines type of the datatype of comparator function/functor:
  - it has default value `less<Key>`,
  - `less` is another functor template implemented in STL,
  - functor `less` overloads operator () where it compares two elements by means of operator <.
- Our simple modification may replace the default functor with function:

  ```
  bool cmp (const string &,const string &)
  ```
- Thus our `set` template instance will be:

  ```
  set<string, bool (*)(const string &,const string &)>
  ```

Remove duplicate words

Template instance is the first step. The types are established, however, the values not:

- `set` instance declares member variable to hold the comparator. In our case the comparator is a function pointer,
- the type is not enough, the member must be initialized,
- the initialization is done in `set` constructor:

```
set ( const Compare & cmp = Compare(),
      const Allocator & alloc = Allocator () );
```

- We need to pass function pointer to our generic instance.
- If we do not pass the pointer, default pointer value will be used:
  - default value will be `nullptr`,
  - thus `set` calls to address `nullptr` and crashes.
- On the other hand the default value works perfectly for functors like the default `less`. Indeed, functor `less` has a default constructor (moreover the instance does not have to be initialized at all since it uses static binding).

Remove duplicate words

```
#include <iostream>
#include <set>
#include <string>
#include <cstring> // strcasecmp
using namespace std;
bool cmp ( const string & a, const string & b )
{
  return strcasecmp ( a . c_str (), b . c_str () ) < 0;
}
int main ( )
{
  set<string,bool(*)(const string&,const string &)> seen ( cmp );
  string str;

  while ( cin >> str )
    if ( seen . insert ( str ) . second )
      cout << str << endl;
  return 0;
}
```

Remove duplicate words

Some people do not like the syntax with the function pointer. In fact the syntax is lengthy (e.g. in function parameters). We may avoid it:

- Use `decltype`.
- Develop our own specialized class `less<string>`, this specialized class will take precedence over the generic version in STL. The advantage is we do not have to modify anything else. The disadvantage is we may break all STL containers that use `less<string>`, all these containers will be case insensitive (break the rest of the program).
- Develop our custom functor similar to `less`.

decltype

- Keyword `decltype` returns type of its argument.
- Only the type is calculated, the expression is not evaluated.
- The result might by a reference type.
- **Note:**

```
int x;
decltype(x) // int
decltype((x)) // int&
```

- `decltype` does not perform function to function pointer conversion.
- We could have used:

```
std::set<std::string, decltype(&cmp)> seen(cmp);
```

or

```
std::set<std::string, decltype(cmp)*> seen(cmp);
```

Remove duplicate words

```
#include <iostream>
#include <set>
#include <string>
#include <cstring> // strcasecmp
using namespace std;
struct iless
{
  bool operator () ( const string & a, const string & b ) const
  { return strcasecmp ( a . c_str (), b . c_str () ) < 0; }
};
int main ( )
{
  set<string,iless> seen;
  string str;

  while ( cin >> str )
    if ( seen . insert ( str ) . second )
      cout << str << endl;
  return 0;
}
```

Remove duplicate words

Display the output sorted alphabetically:

- set values are stored in a tree (red-black tree),
- we get the values sorted when traversing the tree (e.g. by iterator),
- sort order is defined by the comparator used (default is operator <).

The output may be done in a loop iterating over the set, or by `copy` function:

- there is an output iterator similar to input iterator – `ostream_iterator`,
- output iterator may append a string separator after each value displayed,
- we will separate the values by newlines.

Remove duplicate words

```
#include <iostream>
#include <set>
#include <string>
#include <iterator>

using namespace std;

int main ( )
{
  set<string> seen;
  string str;

  while ( cin >> str )
    seen . insert ( str );

  for ( set<string>::iterator it = seen . begin();
          it != seen . end (); ++it ) cout << *it << endl;
  return 0;
}
```

Remove duplicate words

```
#include <iostream>
#include <set>
#include <string>
#include <iterator>
#include <algorithm>
using namespace std;

int main ( )
{
  set<string> seen;
  string str;

  while ( cin >> str )
    seen . insert ( str );

  copy ( seen . begin(), seen . end (),
         ostream_iterator<string> ( cout, "\n" ) );
  return 0;
}
```

Remove duplicate words

Modification: the output shall be sorted case-insensitively but the duplicate removal shall be case-sensitive:

- the set comparator must be case-sensitive,
- we need to change the sort order once the input is read,
- we cannot sort the contents of a set.

We may copy set contents into an array:

- `vector` constructor does the copying,
- `vector` contents may be sorted by `sort` function,
- we need to pass a custom comparator to the `sort` function.

Remove duplicate words

```
...
struct iless {
  bool operator () (const std::string& a,
                    const std::string& b) const {
    return strcasecmp(a.c_str(), b.c_str()) < 0;
  }
};

int main () {
  std::set<std::string> seen;
  for (std::string str; std::cin >> str; seen.insert(str)) ;

  std::vector<std::string> data(seen.begin(), seen.end());

  std::sort(data.begin(), data.end(), iless());
  std::copy(data.begin(), data.end(),
    std::ostream_iterator<std::string>(std::cout, "\n"));

  return 0;
}
```

Lambda functions – motivation

The example code is a standard C++ / STL solution before lambda function were introduced:

- the comparator is provided in class `iless`,
- the function is declared outside of `main`'s scope.

The program works correctly, however, it is not ideal:

- class `iless` is accessible from the outside of function `main`,
- identifier `iless` may collide with other global identifiers (global namespace pollution),
- class `iless` is very short, its declaration overhead is comparable to the body length,
- if `main` and type `std::string` is not public, there may be a problem with encapsulation (we would have to use `friend` or class method to give `iless` access to `std::string`).

Lambda functions

- STL functions often require function or function object as a parameter:
  - STL function `sort` requires comparator,
  - STL function `copy_if` requires unary predicate,
  - thread creation functions require function (code) to start in the new thread,
  - ...
- The above functions are typically short and are used only once (e.g. the comaprators).
- The function may be declared somewhere "outside", however, it is not convenient.
- Lambda functions provide an alternative way to prepare such functions:
  - the function is anonymous,
  - the code of lambda function is placed directly in the parameter,
  - lambda function is used like any other function in the caller (e.g. inside `sort`).

Lambda funkce – remove duplicate words

```
...
int main () {
  std::set<std::string> seen;
  for (std::string str; std::cin >> str; seen.insert(str)) ;

  std::vector<std::string> data(seen.begin(), seen.end());

  std::sort(data.begin(), data.end(),
    [](const std::string& a, const std::string& b) {
      return strcasecmp(a.c_str(), b.c_str()) < 0;
    }
  );

  std::copy(data.begin(), data.end(),
    std::ostream_iterator<std::string>(std::cout, "\n"));

  return 0;
}
```

Lambda functions – filter coordinates

Out function is given an array of coordinates, observer position, and a radius. The output is an array of coordinates that are within the radius of the observer:

- find the distance between the observer and each coordinate in the array,
- compare the distance with the radius,
- include the matching coordinates into the result.

We use generic function `copy_if` to implement most of the problem.

Lambda functions – filter coordinates

```
struct TCoord {
  double m_X, m_Y;
  double distance ( const TCoord & x ) const {
    return sqrt ( ( m_X - x . m_X ) * ( m_X - x . m_X ) +
                  ( m_Y - x . m_Y ) * ( m_Y - x . m_Y ) );
};
bool coordFilter ( const TCoord & x )
{
   ... // ?? pos, radius ??
}
vector<TCoord> foo ( const vector<TCoord> & src,
            const TCoord & pos, double radius )
{
  vector<TCoord> res;
  copy_if ( src . begin (), src . end (),
          back_inserter ( res ), coordFilter );
  return res;
}
```

Lambda functions – filter coordinates

The outlined solution is not correct:

- the testing function needs to know the observer position and the radius,
- the information is known to `foo`,
- we have to pass this information to `coordFilter`.

How to pass additional information:

- global variables – NO (multithreaded programs, colliding identifiers, global variables used by multiple parts of the program, ...),
- a functor – a temporary class implementing overloaded operator `()`,
- a lambda function with capture.

Lambda functions – filter coordinates

```
struct TCoord {
  double m_X, m_Y;
  double distance ( const TCoord & x ) const {
    return sqrt ( ( m_X - x . m_X ) * ( m_X - x . m_X ) +
                  ( m_Y - x . m_Y ) * ( m_Y - x . m_Y ) );
  }
};
struct coordFunctor { // a solution with functor
   coordFunctor ( const TCoord & pos, double r ) :
     m_Pos ( pos ), m_R ( r ) { }
   bool operator () (const TCoord & x ) const {
     double dist = m_Pos . distance ( x );
       // floating point comparison
     return dist - m_R <= 1000 * DBL_EPSILON * m_R;
   }
  private:
   TCoord   m_Pos;
   double m_R;
};
```

Lambda functions – filter coordinates

```
...
vector<TCoord> foo ( const vector<TCoord> & src,
                     const TCoord & pos, double radius )
{
  vector<TCoord> res;

  // a temporary instance of coordFunctor
  copy_if ( src . begin (), src . end (),
    back_inserter (res), coordFunctor ( pos, radius ) );
  return res;
}
```

Lambda functions – filter coordinates

```
struct TCoord {
  double m_X, m_Y;
  double distance ( const TCoord & x ) const {
    return sqrt ( ( m_X - x . m_X ) * ( m_X - x . m_X ) +
                  ( m_Y - x . m_Y ) * ( m_Y - x . m_Y ) );
  }
};
vector<TCoord> foo ( const vector<TCoord> & src,
                     const TCoord & pos, double radius )
{
  vector<TCoord> res;
  // lambda with capture
  copy_if ( src . begin (), src . end (),
    back_inserter (res), [pos, radius] ( const TCoord & x ) {
       double dist = pos . distance ( x );
       return dist - radius <= 1000 * DBL_EPSILON * radius;
    } );
  return res;
}
```

Lambda functions

The general lambda function syntax is:

```
[capture-list] ( formal-parameters ) { lambda-body }
```

`capture-list` is a list of captured variables. Captured variables are propagated from the containing function to the lambda. Captured variables are preserved among lambda invocations:

- `[]` do not capture any variables,
- `[x,y]` capture listed variables by value. The variables are copied from the containing function to lambda context, modifications by the lambda do not affect the source,
- `[&x,&y]` capture listed variables by reference. References are included in the lambda context, thus modifications from the lambda directly propagate to the source,
- `[&x,y]` mixed by reference / by value captures,

Lambda functions

`[&]` capture all variables used in the lambda by reference,

`[=]` capture all variables used in the lambda by value,

`[this]` capture `this` pointer from the containing method.

Note: local variables may be declared in lambda, however, the value of local variables is not preserved among lambda invocations.

Lambda functions

How to develop own function that accepts lambda as a parameter? Generic function `accumulate_if` is an extension of `accumulate`. Only matching values will be included into the resulting value, i.e. we shall pass a predicate to `accumulate_if`:

```
template <class Iter, class T, class Predicate>
T accumulate_if ( Iter st, Iter en, T init, Predicate op )
{
  for ( ; st != en ; ++st )
    if ( op ( *st ) )
      init = init + *st;
  return init;
}
```

The predicate may be any of: function / functor / lambda function.

Lambda functions

- We used a generic parameter in the example function. Thus the compiler deduced the type of `Predicate` based on the actual parameter.
- To store a lambda function in a variable, we may use keyword `auto` to deduce its type.
- If lambda function does not capture any variables, the type of lambda is compatible with function pointer.
- There is a special type designed for lambda functions: `std::function`. The type is declared in header file `functional`.
- A variable of type `std::function` can store any lambda function but it will increase the compilation time and it may lead to slower execution compared to declaring the variable with `auto`.

Sample use of std::function

```
template <class Iter, class T>
T accumulate_if ( Iter st, Iter en, T init,
                  function<bool(const T&)> op )
{
  for ( ; st != en ; ++st )
    if ( op ( *st ) )
      init = init + *st;
  return init;
}
```

Lambda functions

```
void foo ( vector<int> & src, int threshold ) {
  auto filter = [&threshold] ( int val ) {
    return val >= threshold;
  };

  vector<int> a;
  copy_if ( src . begin(), src . end (),
            back_inserter ( a ), filter );

  threshold /= 2;
  vector<int> b;
  copy_if ( src . begin(), src . end (),
            back_inserter ( b ), filter );
  ...
}
```

Lambda functions

How to compile lambda functions?

- The compiler automatically creates a new functor class:
  - it declares member variables for captured variables,
  - constructor initializes members with the captured values,
  - there is overloaded operator `()`, the body corresponds to the lambda,
  - such overloaded operator is implicitly `const`, thus lambda cannot modify variables captured by value (this may be changed with `mutable` declaration),
  - return type is deduced based on the type used in `return` statements. Programmer may specify return type explicitly using trailing return type syntax.
- If the lambda does not capture any variables, a conversion operator to a function pointer is generated.

Lambda functions

```
vector<int> seq;
int start = 123;

// error
generate_n ( back_inserter ( seq ), 100,
             [start] () { return start ++; } );

// ok
generate_n ( back_inserter ( seq ), 100,
             [start] () mutable { return start ++; } );
// start = 123

// ok
generate_n ( back_inserter ( seq ), 100,
             [&start] () { return start ++; } );
// start = 223
```

Lambda functions

```
TCoord center ( 10, 20 );

// error, inconsistent types
auto filter1 = [&] ( const TCoord & x ) {
  if ( x . distance ( center ) < 1e-4 )
    return 0;
  else
    return x . distance ( center );
};

// ok, -> double
auto filter2 = [&] ( const TCoord & x ) -> double {
  if ( x . distance ( center ) < 1e-4 )
    return 0;
  else
    return x . distance ( center );
};
```

STL – overview

STL library is built on generic classes. These are used to implement:

- data structures (vector, set, map, ...),
- auxiliary structures (pair, tuple, ...),
- internal structures used by STL (iterators, allocators, ...).

Data structures usually have several generic parameters. Less common generic parameters have default values to simplify development, e.g.:

```
template <class T, class Allocator = std::allocator<T> >
vector;
```

- generic parameter `T` is the type of vector elements,
- parameter `Allocator` denotes the class that shall manage memory allocations. The default allocator `std::allocator` uses heap to allocates the memory,
- the programmer may supply his own allocator (comparator, hash function, ...) to change the default behavior.

STL containers

- A container (collection) is an abstract data type to organize the data in some well-defined manner. The library provides a variety of common containers.
- The means of element access and the time complexity to access elements varies widely among containers.
- All STL containers provide `public` copy constructor and assignment operator =.

STL containers – array

- An encapsulated array of some fixed size. Supports index range checks and copying,
- `#include <array>`, since C++ 11,
- `RandomAccessIterator`.

```
array<int, 10> x;
x . fill ( -1 );
x[3] = 100; // unchecked access
array<int, 10> y(x);
for ( array<int,10>::size_type i = 0; i < x . size (); i ++ )
  x[i] += 100;
copy ( x . begin(), x . end (),
       ostream_iterator<int> ( cout, "\n" ) );

array<int,20> z(x);  // !!!
```

STL containers – vector

- An encapsulated resizing array. Supports index range checks, copying, resizing,
- element access time: $O(1)$,
- `#include <vector>`,
- `RandomAccessIterator`.

```
vector<int> x;
x . resize ( 10 ); // 0 0 0 ... 0
x . push_back ( 100 );
x . insert ( x . begin() + 5, 200 ); // make room
x[3] = 120; // overwrite, no index check
x . at (4) = 150; // overwrite, check index
copy ( x . begin(), x . end (),
       ostream_iterator<int> ( cout, "\n" ) );
// copy range
vector<int> y ( x . begin () + 1, x . begin () + 9 );
sort ( y . begin(), y . end () );
copy ( y . begin(), y . end (),
       ostream_iterator<int> ( cout, "\n" ) );
```

STL containers – deque

- A doubly-ended queue. Support to add/remove elements from both start/end in $O(1)$ time, element access (indexing) in $O(1)$ time,
- `#include <deque>`,
- `RandomAccessIterator`,
- deque can be used in place of `stack` and `queue`.

```
deque<int> x;
x . push_back ( 100 ); x . push_front ( 200 );
x . push_back ( 300 ); x . push_front ( 400 );
x . insert ( x . begin() + 1, 500 );
// copy, read, display & pop
for ( deque<int> y (x); ! y . empty () ; y . pop_front () )
  cout << y . front () << endl;

x . erase ( x . begin () + 1, x . begin () + 3 );
// iterate in reverse direction
for ( deque<int>::reverse_iterator it = x . rbegin();
        it != x . rend (); ++it )
  cout << *it << endl;
```

STL containers – `list`

- A doubly-linked list. Add/remove elements from arbitrary position (start/end/iterator position) in $O(1)$ time,
- `#include <list>`,
- `BidirectionalIterator`.

```
list<int> x;
x . push_back ( 100 );
x . push_back ( 200 );
x . push_front ( 300 );
list<int>::iterator pos = x . begin ();
pos ++;
x . insert ( pos, 400 );
x . erase ( pos + 1 ); // !!!
pos ++;
x . erase ( pos );
// iterate forward
for ( list<int>::iterator it = x . begin();
        it != x . end (); ++it )
  cout << *it << endl;
```

STL containers – set

- A set of elements (element is either present or not present). Insert element, remove element, test element presence in $O(\log n)$ time,
- `#include <set>`,
- `BidirectionalIterator`.

```
set<int> x;
x . insert ( 20 );
x . insert ( 100 );
x . insert ( 1000 );
cout << ( x . count ( 20 ) == 1 ? "present" : "not present" )
     << endl;

set<int>::iterator pos = x . find ( 1000 );
if ( pos != x . end () ) x . erase ( pos );

for ( set<int>::iterator it = x . begin();
        it != x . end (); ++it )
  cout << *it << endl;
```

STL containers – map

- A map (key - value table). Insert element, remove element, access element in $O(\log n)$ time,
- `#include <map>`,
- `BidirectionalIterator`.

```
map<string,int> x;
x . insert ( make_pair ( "test", 10 ) );
x["key"] = 20;
x["testkey"] = x["test"] + x["key"];

for ( map<string,int>::const_iterator it = x . begin();
        it != x . end (); ++it )
  cout << it -> first << "->" << it -> second << endl;

map<string,int>::iterator pos = x . find ( "test" );
cout << (  pos != x . end () ? "present"
        : "not present" ) << endl;

x . erase ( pos );
```

Further STL containers

`forward_list`:

- single-linked list. A variant of `list` with decreased memory overhead. Some operations are not available (e.g. insert before iterator).
- `#include <forward_list>`, C++ 11
- `ForwardIterator`.

`stack`:

- a stack, implemented as a `deque`, `list`, or vector wrapper, interface reduced to LIFO,
- `#include <stack>`
- no iterator.

`queue`:

- a queue implemented as a `deque` or `list` wrapper, interface reduced to FIFO,
- `#include <queue>`
- no iterator.

Further STL containers

`priority_queue`:

- a queue with priorities. The priority is achieved by internal heap organization. Both insert and remover requires $O(\log n)$ time,
- `#include <queue>`
- no iterator.

`multiset`, `multimap`:

- classes correspond to `set` and `map`,
- more instances with the same key may be included in the container,
- `#include <set>` / `#include <map>`
- `BidirectionalIterator`.

`bitset`:

- a fixed-size array of `bool` values,
- memory-efficient implementation (uses individual bits),
- `#include <bitset>`.

Further STL containers

`unordered_set`, `unordered_multiset`,`unordered_map`, `unordered_multimap`:

- equivalents of sets/maps, implemented by means of hash tables,
- insert/delete/access in average $O(1)$ time,
- STL library implements hash functions for primitive data types,
- a custom hash function must be supplied when using own types,
- `#include <unordered_set>`, `#include <unordered_map>`, since C++ 11,
- `ForwardIterator`.

Further information on STL containers:

- http://en.cppreference.com/w/,

Iterators

- STL uses iterators to:
  - traverse containers,
  - locate some specific position in a container (e.g. where element shall be inserted/removed),
  - to define a range of values to process by some function (sort, sum, copy, ...),
  - to search in a container.
- Iterators are usually class templates that implement overloaded operators. Their behavior is similar to pointers:

  `*` – dereference (element access),

  `==` and `!=` – compare iterators (same/different position),

  `++` – advance forward (both prefix and postfix version, prefix is often more efficient).
- Containers provide methods `begin()` / `end()` to return iterators pointing to the first element / one past the last element,
- Some containers do not use iterators: `stack`, `queue`, `priority_queue`.

Iterators

InputIterator read dereference, move forward, and test for (in)equality. Dereferenced data cannot be read again. Example: `istream_iterator`.

OutputIterator write dereference, move forward, and test for (in)equality. Once written the data cannot be overwritten. Example: `ostream_iterator`.

ForwardIterator interface of `InputIterator`, read/write dereference, can dereference the same position more than once. Example: `forward_list`.

BidirectionalIterator interface of `ForwardIterator` and move backward (operator `--`). Examples: `list`, `set`, `map`, ...

RandomAccessIterator interface of `BidirectionalIterator` plus move forward/backward by any number of positions (operators += and -=), indexing operator `[n]`, iterator difference, relational operators (<, ...). Examples: `vector`, `deque`.

Iterators

`X` is iterator type, `a` and `b` are instances of that iterator, `t` is an object referenced by the iterator, and `n` is an integer.

category	operation	code
All	copy constructor	`X b(a);`
All	assign	`b = a;`
All	Preincrement	`++a`
All	Postincrement	`a++`
Rand, Bidir, Forw, Inp	equality	`a == b`
Rand, Bidir, Forw, Inp	inequality	`a != b`
Rand, Bidir, Forw, Inp	r-value dereference	`*a` / `a -> m`
Rand, Bidir, Forw, Out	l-value dereference	`*a = t`
	(non-const iterators)	`a -> m = t`
		`*a++ = t`

Iterators

category	operation	code

Rand, Bidir, Forw	multi-pass: can dereference (and increment) the same position multiple times	`b=a;`
		`t1 = *a++;`
		`t2 = *b;`

Rand, Bidir	decrement	`--a`
		`a--`
		`*a--`

Rand	add/subtract integer	`a + n`
		`a - n`
		`n + a`

Rand	iterator difference	`a - b`

Rand	relational op $<, <=, >, >=$	`a < b`
		`a <= b`
		`a > b`
		`a >= b`

Rand	advance by `n`	`a += n` `a -= n`
Rand	indexing	`a[n]`

STL – generic functions

- STL library provides a set of generic functions to simplify typical operations with collections: traversal, searching, aggregations, ...
- the functions are generic (templates),
- the function usually take iterator parameters to define the range of elements to process,
- programmer supplied functions / functors are often required to define the operation with the elements,
- the generic functions are accessible via `#include <algorithm>` and `#include <numeric>`,
- we will present some typical functions.

STL – generic functions

`for_each(st,en,fn)` – call function/functor `fn` for all elements in range `st` to `en`,

`find(st,en,x)` – find first occurrence of `x` in the range `st` to `en`,

`find_if(st,en,fn)` – find first element in the range `st` to `en`, such that predicate `fn` is true,

`copy(st,en,dst)` – copy elements from the range `st` to `en` into destination position `dst` (and on). The exact behavior may be changed by the choice of `dst` – input/output, move, append, ...

`copy_if(st,en,dst,fn)` – copy like `copy`, apply only to elements where predicate `fn` is true,

`transform(st,en,dst,fn)` – copy like `copy`, the elements are transformed by function/functor `fn` before writing,

`remove_if(st,en,fn)` – remove elements in the range `st` to `en`, only remove elements where predicate `fn` is true,

`sort(st,en,fn)` – sort elements in the range `st` to `en`, the comparison is defined by comparator `fn`.

STL – generic functions

Other common tasks and the corresponding STL functions:

- binary search: `lower_bound`, `upper_bound`, `binary_search`,
- merging and operations based on merging (intersection, union of sorted arrays/sets): `merge`, `set_union`, `set_intersection`,
- heap data structure support: `make_heap`, `push_heap`, `pop_heap`,
- minimum and maximum: `min_element`, `max_element`,
- aggregation: `accumulate`,
- test properties of all elements: `all_of`, `any_of`, `none_of`.

Further information on generic STL algorithms:

- http://en.cppreference.com/w/,

Questions and answers

Questions ...